

Point Cloud Rendering

One of the problems we're facing is trying to render large point cloud data sets on the web — a 1 km² data set contains >600k points of data, and we will have potentially hundreds of these tiles in the final map.

Part A)

Your objective is to write an algorithm that will display just some of these points at different map scales. So, if you are at the highest level of zoom, all data will be displayed. But at the 2nd level of zoom, maybe every other point will be displayed. So on and so forth through 10 layers of zoom. This will be zero indexed, so the 0th layer corresponds to 0 “missing” points, 1 to 1, 2 to 2, etc.

Ex: 0th layer

Ex: 1st layer . - . - . -

Ex: 2nd layer . -- . -- . --

Keep in mind that this is a 3D data set... some dimensions may be more important than others. Each data point contains a longitude, latitude, and elevation.

The expected output should be a function that takes in a 2D array with dimensions $3 \times X$ where X is the total number of points as well as a zoom modification, and returns another (smaller) 2D array with dimensions $3 \times Y$ where Y is the new number of points. I will provide you with an example data-set.

The final number of points you will have will be

$$Y = \left(\frac{X}{1+z}\right)$$

Where $z \in (0, 1, 2, \dots, 10)$ and represents the zoom level.

```
1 example_dataset = [  
2     [123456789.0123456,   -987654321.0987654,   1538461538.2469132],  
3     [-654321987.6543210,   987654321.9876543,   246913427.3747485],  
4     [7533692187.6538646,  -2147483647.0000000,   1111590261.9647413]  
5 ]
```

This algorithm should be data-type agnostic: What this means, is it should work with integers, binary, more complex objects, etc.

Part B)

What is the time complexity of your algorithm? Time complexity refers to how the time the algorithm takes scales with the size of the data set. For example, constant time means that it takes the same amount of time regardless of the size — linear time means there is a one to one relationship between size of dataset and time, and exponential means as the data set grows linearly, the time it takes grows exponentially. You should aim for a linear time complexity if possible — this is the ideal for an algorithm like this without making it unnecessarily complex. Exponential would be bad. It is worth doing some independent research on time complexity before or after doing this problem because it is an important concept to understand in computer science.